

BEE 4750 Homework 3: Dissolved Oxygen and Monte Carlo

Name: Evelyn Chen and Natalie Ho

ID: ec877, nh424

Due Date

Thursday, 10/16/25, 9:00pm

Overview

Instructions

- Problem 1 asks you to implement a model for dissolved oxygen in a river with multiple waste releases and use this to develop a strategy to ensure regulatory compliance.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
import Pkg
Pkg.activate(@__DIR__)
Pkg.instantiate()
```

Activating project at `~/BEE\_4750/hw3-natalie-evelyn`

```
using Random
using Plots
using LaTeXStrings
using Distributions
```

Problems (Total: 30 Points)

Problem 1 (30 points)

A river which flows at 6 km/d is receiving waste discharges from two sources which are 15 km apart. The oxygen reaeration rate is 0.55 day⁻¹, and the decay rates of CBOD and NBOD are 0.35 and 0.25 day⁻¹, respectively. The river's saturated dissolved oxygen concentration is 10 mg/L.

If the characteristics of the river inflow and waste discharges are given in Table 1, write a Julia model to compute the dissolved oxygen concentration from the first wastewater discharge to an arbitrary distance d km downstream. Use your model to compute the minimum dissolved oxygen concentration up to 50 km downstream and how far downriver this maximum occurs.

Parameter	River Inflow	Waste Stream 1	Waste Stream 2
Inflow	100,000 m ³ /d	10,000 m ³ /d	15,000 m ³ /d
DO Concentration	7.5 mg/L	5 mg/L	5 mg/L
CBOD	5 mg/L	50 mg/L	45 mg/L
NBOD	5 mg/L	35 mg/L	35 mg/L

Table 1: River inflow and waste stream characteristics for Problem 1.

Problem 1.1

Implement the Streeter-Phelps (analytic) solution for the dissolved oxygen concentration. Plot the dissolved oxygen concentration from the first waste stream to 50 km downriver. What is the minimum value in mg/L?

```
using Plots
using LaTeXStrings

function do_simulate(x, C0, B0, N0, ka, kc, kn, Cs, U)
    B = B0 * exp(-kc * x / U)
    N = N0 * exp(-kn * x / U)

    1 = exp(-ka * x / U)
    2 = (kc/(ka-kc)) * (exp(-kc * x / U) - exp(-ka * x / U))
    3 = (kn/(ka-kn)) * (exp(-kn * x / U) - exp(-ka * x / U))

    C = Cs * (1 - 1) + (C0 * 1) - (B0 * 2) - (N0 * 3)
    return (C, B, N)
```

```

end

U, ka, kc, kn = 6.0, 0.55, 0.35, 0.25
Cs = 10.0

Qr, Q1, Q2 = 100_000.0, 10_000.0, 15_000.0
Cr, Br, Nr = 7.5, 5.0, 5.0
Cw1, Bw1, Nw1 = 5.0, 50.0, 35.0
Cw2, Bw2, Nw2 = 5.0, 45.0, 35.0

x_split, x_end, Δx = 15.0, 50.0, 0.05
x1      = 0.0:Δx:x_split
x2local = 0.0:Δx:(x_end - x_split)

# Point 1: mix river + Waste 1 at x = 0
Qtot1 = Qr + Q1
C01 = (Qr*Cr + Q1*Cw1) / Qtot1
B01 = (Qr*Br + Q1*Bw1) / Qtot1
N01 = (Qr*Nr + Q1*Nw1) / Qtot1

do_out1 = ( -> do_simulate( , C01, B01, N01, ka, kc, kn, Cs, U)).(x1)
C1 = [d[1] for d in do_out1]
B1 = [d[2] for d in do_out1]
N1 = [d[3] for d in do_out1]

C15m = C1[end]
B15m = B01 * exp(-kc * x_split / U)
N15m = N01 * exp(-kn * x_split / U)

# Point 2: mix with Waste 2 at x = 15
Qtot2 = Qtot1 + Q2
C02 = (Qtot1*C15m + Q2*Cw2) / Qtot2
B02 = (Qtot1*B15m + Q2*Bw2) / Qtot2
N02 = (Qtot1*N15m + Q2*Nw2) / Qtot2

do_out2 = ( -> do_simulate( , C02, B02, N02, ka, kc, kn, Cs, U)).(x2local)
C2 = [d[1] for d in do_out2]
B2 = [d[2] for d in do_out2]
N2 = [d[3] for d in do_out2]

# Concatenate full profile
x = vcat(collect(x1), x_split .+ collect(x2local))

```

```

C = vcat(C1, C2)
B = vcat(B1, B2)
N = vcat(N1, N2)

# Minimum DO
imin = argmin(C)
xmin = x[imin]
Cmin = C[imin]
println("Minimum DO = $(round(Cmin,digits=3)) mg/L at x = $(round(xmin,digits=2)) km")

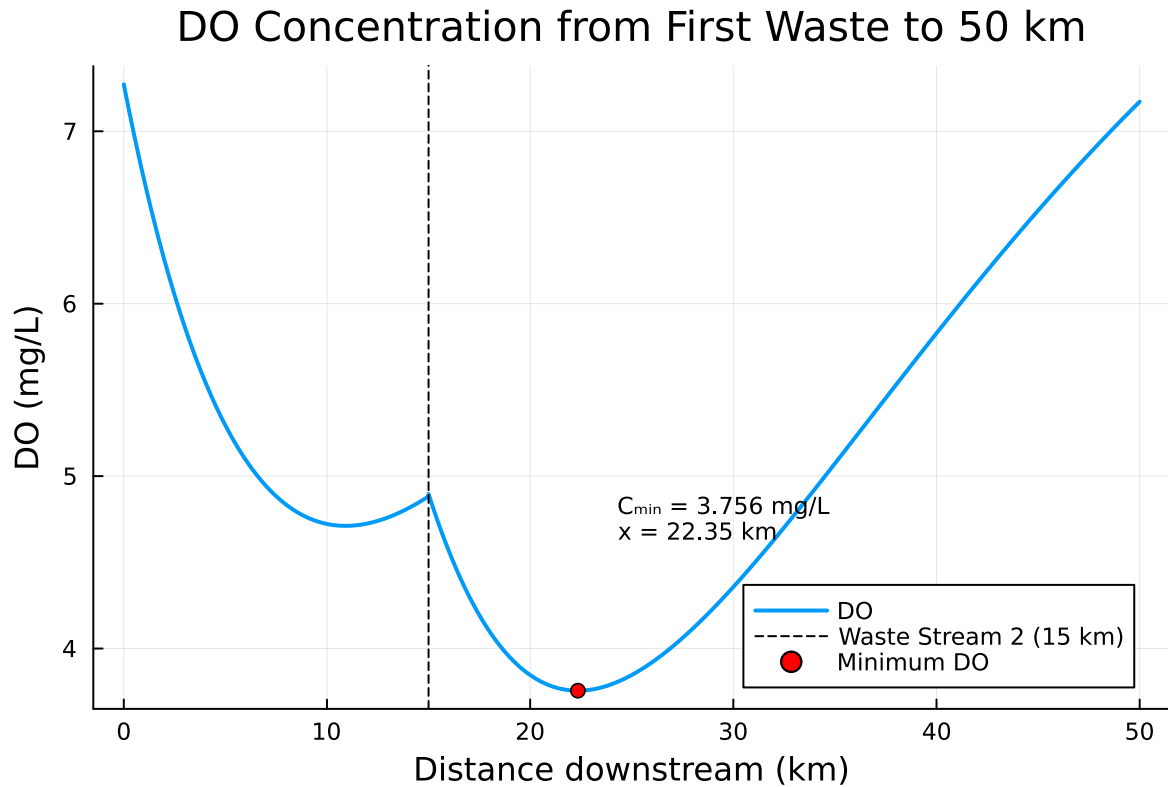
# Plot DO from first waste to 50 km
p = plot(
    x, C,
    label = "DO",
    xlabel = "Distance downstream (km)",
    ylabel = "DO (mg/L)",
    title = "DO Concentration from First Waste to 50 km",
    lw = 2,
)
vline!(p, [x_split], label = "Waste Stream 2 (15 km)", linestyle = :dash, color = :black)

# Mark minimum DO and place annotation so it doesn't overlap the legend/marker
scatter!(p, [xmin], [Cmin], label = "Minimum DO", color = :red, markerstrokecolor = :black)
annotate!(p, (xmin + 2, Cmin + 1.0, text("C   = $(round(Cmin, digits=3)) mg/L\nx = $(round(xmin, digits=2)) km")

display(p)

```

Minimum DO = 3.756 mg/L at x = 22.35 km



This code simulates how dissolved oxygen (DO) changes along a river that receives two separate waste discharges and then plots the results. It starts by loading `Plots` and `LaTeXStrings`, which I used to make the graph and format the labels nicely. The main function, `do_simulate`, and Streeter Phelps Equation models how DO, CBOD, and NBOD change as water moves downstream. Both CBOD and NBOD decay exponentially based on their rate constants (k_c and k_n), while oxygen from the atmosphere is added back into the system through the reaeration constant (k_a). The terms (, ,) describe how these processes interact which shows how oxygen is being lost and replaced along the way. The final DO (C) at any point is a balance between atmospheric oxygen being added, oxygen carried from upstream, and oxygen used up by CBOD and NBOD decay.

After defining that function, I set all the constants and conditions for the river. This includes things like mean velocity (U), rate constants (k_a , k_c , k_n), and the saturated DO level (C_s). I then set the flow rates and concentrations for the main river and two different waste streams. The first one enters at 0 km and the second at 15 km downstream. At each mixing point, I calculated flow averages to find the new combined concentrations of DO, CBOD, and NBOD. I first simulated the stretch from 0 to 15 km (after Waste 1), then updated those conditions at 15 km, added in Waste 2, and simulated the rest of the river up to 50 km.

After both sections were simulated, I joined them together into full profiles for DO, CBOD, and NBOD across the entire river length. I then found where the DO reached its lowest value

which is the critical point where oxygen is most depleted. Finally, I plotted DO versus distance, added a dashed line at the second waste input (15 km), and marked the minimum DO with a red point and an annotation showing its value and location. Overall, showing how oxygen levels drop and recover after each waste input, showing where along the river conditions are most stressed.

Problem 1.2

Implement a numerically-integrated (discretized) solution for the dissolved oxygen concentration. Using a resolution of 0.5km, conduct a simulation and plot the results on the same axis as the plot from Problem 1.1. How has the minimum value changed? What do you attribute this difference to (be specific about the source of the difference(s) in terms of the simulation dynamics, not just that one is a numerical approximation).

The minimum DO, as calculated by the analytical model, is 3.756 mg/L at $x = 22.35$ km. On the other hand, the numerical model computes the minimum DO as 3.804 mg/L at $x = 22.5$ km. There is a slight difference, of approximately 0.048 mg/L, as well as a difference of 0.15 km in terms of downstream locations of the respective minimum DOs.

As for why we might see these differences, the two solutions use different simulation methods. Numerical integration has a greater uncertainty than analytic calculation (which uses exact closed-form exponentials), since our resolution ($\text{res} = 0.5$) isn't negligible. This is especially critical at the location of instantaneous mixing: if the stepwise dx does not calculate exactly for 15.0 km (the location of the second waste stream), and rather slightly after 15.0km, this would lead to a slight rounding error.

```
# first, given parameters
v = 6.0 # km/d
k2 = 0.55 # 1/d
k1_cbod = 0.35 #1/d
k1_nbod = 0.25 #1/d
DO_sat = 10.0 #mg/L

# River Inflow
Qr, DOr, CBODr, NBODr = 100_000.0, 7.5, 5, 5

# Waste Stream 1
Qw1, DOW1, CBODw1, NBODw1 = 10_000.0, 5, 50, 35

# Waste Stream 2
Qw2, DOW2, CBODw2, NBODw2 = 15_000.0, 5, 45, 35
```

```

xw1 = 0.0 # starting point/origin
xw2 = 15.0

# Assuming conservative, instantaneous mixing
function Cmix(Q1, C1, Q2, C2)
    Q = Q1 + Q2
    if Q != 0.0
        return (Q1*C1 + Q2*C2) / Q
    end
    return 0.0
end

# concentrations @ x=0 (waste stream 1)
Q1 = Qr + Qw1
DO_w1 = Cmix(Qr, DOr, Qw1, DOW1)
CBOD_w1 = Cmix(Qr, CBODr, Qw1, CBODw1)
NBOD_w1 = Cmix(Qr, NBODr, Qw1, NBODw1)

# DO deficit immediately after waste stream 1
DO_1 = DO_sat - DO_w1

# Two equations:
# 1) Exact exponential decay for L:  $L(t+dt) = L(t)*\exp(-k_1dt)$ 
# 2) Forward Euler for DO deficit:  $dD/dt = -k_2*D + k_1_{cbod}*L_1 + k_1_{nbod}*L_2$ 

function DO_numeric(xmax, res=0.5)
    x_array = 0:res:xmax # step size res, 0-xmax
    n = length(x_array)
    DO_num, L1_num, L2_num = zeros(n), zeros(n), zeros(n)

    # @ x=w1 (after first mixing)
    DO_num[1] = DO_w1
    L1_num[1] = CBOD_w1
    L2_num[1] = NBOD_w1
    Q = Qr + Qw1

    # @ x=w2)
    # Exact exponential decay for L:  $L(t+dt) = L(t)*\exp(-k_1dt)$ 
    for i in 2:n
        xprev = x_array[i-1]
        xcurr = x_array[i]
        dt = (xcurr - xprev)/v

```

```

L1 = L1_num[i-1] * exp(-k1_cbod*dt)
L2 = L2_num[i-1] * exp(-k1_nbod*dt)

# Forward Euler for DO deficit: dD/dt = -k2*D + k1_cbod*L1 + k1_nbodL2
D_prev = DO_sat - DO_num[i-1]
dD_dt = -k2 * D_prev + k1_cbod * L1 + k1_nbod * L2
DO_new = DO_sat - (D_prev + dD_dt * dt)

# instant mixing @ location where xw1 and xw2 combine (using Q and Q_w2)
if xprev < xw2 && xcurr >= xw2
    Q_before = Qr + Qw1
    DO_mixed = Cmix(Q_before, DO_new, Qw2, DOw2)
    # For the L's, L1 and L2 above represent the decayed values at the mixing point
    CBOD_mixed = Cmix(Q_before, L1, Qw2, CBODw2)
    NBOD_mixed = Cmix(Q_before, L2, Qw2, NBODw2)
    DO_new = DO_mixed
    L1 = CBOD_mixed
    L2 = NBOD_mixed
    Q = Q_before + Qw2
end
DO_num[i] = DO_new
L1_num[i] = L1
L2_num[i] = L2
end

return x_array, DO_num, L1_num, L2_num
end

# compute the minimum dissolved oxygen concentration up to 50 km downstream (res = 0.5)
x_vals, DO_vals, L1_vals, L2_vals = DO_numeric(50.0, 0.5)

# find minimum DO
min_DO = minimum(DO_vals)
# find how far downriver this minimum occurs
x_min = x_vals[argmin(DO_vals)]

println("Minimum DO =", min_DO, " mg/L at x =", x_min, " km downstream")

# PLOT
p = plot(
    x_vals, DO_vals,
    label = "DO",

```



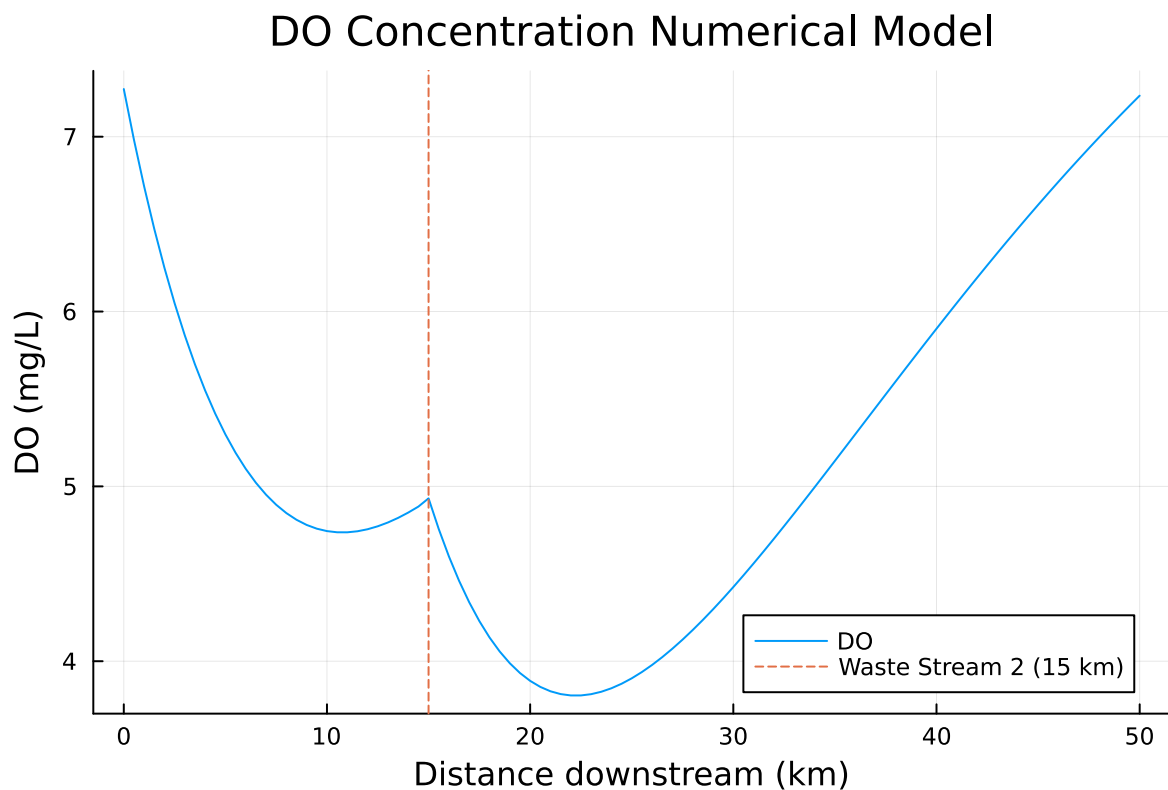
```

xlabel = "Distance downstream (km)",
ylabel = "DO (mg/L)",
title = "DO Concentration Numerical Model",
)

# 15km identification (xw2)
vline!(p, [xw2], label = "Waste Stream 2 (15 km)", linestyle = :dash)
display(p)

```

Minimum DO = 3.804239855608298 mg/L at x = 22.5 km downstream



Problem 1.3

Using the analytic model, what is the minimum level of treatment (% removal of organic waste; assume this is equivalent to the same level of reduction of the CBOD and NBOD) for waste stream 1 that will ensure that the dissolved oxygen concentration is in compliance with the 4 mg/L standard along with a 5% margin of safety, assuming that waste stream 2 remains untreated? How about if only waste stream 2 is treated?

```

# I started by defining the criteria my river model had to meet the dissolved oxygen (DO)
# needed to stay above 4 mg/L everywhere downstream. Since regulations often include
# a safety buffer, I added the 5% margin to make it 4.2 mg/L.
C_threshold = 4.0 * 1.05 # 4 mg/L with 5% margin = 4.2 mg/L

# Then I wrote a function that let me test "what if" scenarios for different levels
# of wastewater treatment. I wanted to see how much CBOD and NBOD removal (the two
# pollutants that consume oxygen) I needed to reach compliance.
function minDO_percent(p1, p2)
    # I began by saving the original values of CBOD, NBOD, and DO for both waste streams.
    # That way I could restore everything back to its starting condition after running each t
    CBODw1_orig, NBODw1_orig = CBODw1, NBODw1
    CBODw2_orig, NBODw2_orig = CBODw2, NBODw2
    CBOD_w1_orig, NBOD_w1_orig, DO_w1_orig = CBOD_w1, NBOD_w1, DO_w1

    # I applied treatment by reducing each waste's CBOD and NBOD by a percentage p.
    # p1 was the treatment applied to Waste 1 and p2 to Waste 2.
    # I kept the dissolved oxygen of the wastes the same since treatment doesn't change DO d
    global CBODw1 = CBODw1_orig * (1 - p1)
    global NBODw1 = NBODw1_orig * (1 - p1)
    global CBODw2 = CBODw2_orig * (1 - p2)
    global NBODw2 = NBODw2_orig * (1 - p2)

    # Once I adjusted the waste concentrations, I recalculated the mixture at x=0 km,
    # where Waste Stream 1 entered the river. I used the Cmix function to blend
    # the river inflow with the waste flow based on their discharge rates.
    global DO_w1 = Cmix(Qr, DOr, Qw1, DOw1)
    global CBOD_w1 = Cmix(Qr, CBODr, Qw1, CBODw1)
    global NBOD_w1 = Cmix(Qr, NBODr, Qw1, NBODw1)

    # After resetting the initial conditions, I ran my existing DO_numeric model,
    # which simulated how DO, CBOD, and NBOD changed along 50 km of river flow.
    _, DO_vals, _, _ = DO_numeric(50.0, 0.5)

    # I found the minimum DO concentration along the river to check if it met the 4.2 mg/L t
    cmin = minimum(DO_vals)

    # Once the test finished, I restored all the original parameter values
    # so the next simulation started from the same conditions.
    global CBODw1, NBODw1 = CBODw1_orig, NBODw1_orig
    global CBODw2, NBODw2 = CBODw2_orig, NBODw2_orig
    global CBOD_w1, NBOD_w1, DO_w1 = CBOD_w1_orig, NBOD_w1_orig, DO_w1_orig

```

```

    # Finally, I returned the minimum DO value for that treatment scenario.
    return cmin
end

# After that, I created a function that automatically found the minimum treatment
# needed to meet the DO standard. I used a bisection search method, where I started
# with 0% and 100% removal and kept narrowing the range until the model's minimum DO
# was just above 4.2 mg/L.
function find_min_removal_on(stream::Int; tol=1e-4, it=40)
    lo, hi = 0.0, 1.0 # I began with a full range of possible treatment levels (0-100%)

    for _ in 1:it
        mid = (lo + hi)/2 # Each loop, I tested the midpoint between the bounds

        # Depending on which waste stream I was testing,
        # I treated only Waste 1 (p1 = mid, p2 = 0) or only Waste 2 (p1 = 0, p2 = mid).
        cmin = (stream == 1) ? minDO_percent(mid, 0.0) : minDO_percent(0.0, mid)

        # If the DO met the standard, I knew I could possibly get away with less treatment,
        # so I lowered the upper bound. If it didn't meet the standard, I increased the lower bound.
        (cmin >= C_threshold) ? (hi = mid) : (lo = mid)

        # Once the bounds were close enough (within tolerance), I stopped the search.
        (hi - lo) < tol && break
    end

    # By the end, hi represented the smallest fraction of treatment that kept DO compliant.
    p = hi

    # I then reran the model one last time at that treatment level to confirm the final minimum DO.
    c = (stream == 1) ? minDO_percent(p, 0.0) : minDO_percent(0.0, p)

    # I returned both the treatment percentage and the minimum DO found at that level.
    return p, c
end

# Once both functions were ready, I used them to test two cases:
# First, I treated Waste Stream 1 and left Waste Stream 2 untreated.
p_w1, c_w1 = find_min_removal_on(1)

# Then, I treated only Waste Stream 2 while keeping Waste Stream 1 untreated.
p_w2, c_w2 = find_min_removal_on(2)

```

```
# Finally, I printed out all the results.
# This showed the compliance target, the minimum percent removal required for each waste stream
# and the corresponding minimum DO concentration after treatment.
println("Compliance threshold (4 mg/L + 5% MOS): ", C_threshold, " mg/L")
println("Min % removal for Waste 1 only: ", round(100*p_w1, digits=2), "% → min DO = ", round(C_threshold, digits=2), " mg/L")
println("Min % removal for Waste 2 only: ", round(100*p_w2, digits=2), "% → min DO = ", round(C_threshold, digits=2), " mg/L")
```

```
Compliance threshold (4 mg/L + 5% MOS): 4.2 mg/L
Min % removal for Waste 1 only: 24.82% → min DO = 4.2 mg/L
Min % removal for Waste 2 only: 19.36% → min DO = 4.2 mg/L
```

Problem 1.4

Suppose you are responsible for designing a waste treatment plan for discharges into the river, with a regulatory mandate to keep the dissolved oxygen concentration above 4 mg/L. Discuss whether you'd opt to treat waste stream 2 alone or both waste streams equally. What other information might you need to make a conclusion, if any?

To keep the DO concentration above 4 mg/L, I would opt to treat waste stream 2 more than waste stream 1. It is true that treating the discharge from waste stream 1 could prevent the oxygen sag earlier and thus reduce downstream impacts from waste stream 2. However, it can be seen from the graphs that from both methods, the first DO sag (from waste stream 1) doesn't reach 4 mg/L at all, while the global minimum DO occurs after waste stream 2. In addition, calculating and comparing the mass BOD loadings for both waste streams:

$$Qw1 * (CBOD_w1 + NBOD_w1) = 10,000 * (50 + 35) = 850,000mg.$$

$$Qw2 * (CBOD_w2 + NBOD_w2) = 15,000 * (45 + 35) = 1,200,000mg$$

It can be seen that waste stream 2 contributes more BOD mass. Other information that could be useful in making a more informed conclusion could be the costs for each plant (if there are any differences), potential DO variability due to seasonal temperature and flow differences, as well as circumstantial environments around both waste streams (eg. which aquatic habitat is more sensitive?).

Problem 1.5

Suppose that it is known that the DO concentrations at the river inflow can vary according to a LogNormal(2.0, 0.15) distribution. Conduct a Monte Carlo simulation of the DO concentration in the river. If you treat only waste stream 1 (based on your analysis from Problem 1.3), what is the expected probability and 95% confidence interval that the river fails to comply with the regulatory standard of 4 mg/L? How did you decide that your Monte Carlo sample size was sufficiently large?

```
N = 50_000 # number of samples in our Monte Carlo simulation
Random.seed!(1) # reproducible random seed, just set arbitrarily at 1

# inflow = Lognormal(2.0, 0.15)
inflow_DO = LogNormal(2.0, 0.15)

# from Prob 1.3, save:
DOr_orig = DOr
CBODw1_orig, NBODw1_orig = CBODw1, NBODw1
CBOD_w1_orig, NBOD_w1_orig, DO_w1_orig = CBOD_w1, NBOD_w1, DO_w1

count_fails = 0
mins = zeros(N)

# we only want to treat waste stream 1
cbodw1_treat = CBODw1_orig * (1.0 - p_w1)
nbodw1_treat = NBODw1_orig * (1.0 - p_w1)

for i in 1:N
    # sample from the lognormal distribution
    DOr_sample = rand(inflow_DO)

    # update global variables for this iteration
    DOr = DOr_sample
    CBODw1 = cbodw1_treat
    NBODw1 = nbodw1_treat

    # update mixed conditions
    DO_w1 = Cmix(Qr, DOr, Qw1, DOw1)
    CBOD_w1 = Cmix(Qr, CBODr, Qw1, CBODw1)
    NBOD_w1 = Cmix(Qr, NBODr, Qw1, NBODw1)

    # run the model downstream
    x_vals_mc, DO_vals_mc, _, _ = DO_numeric(50.0, 0.5)
```

```

    # Record min DO and check to match with compliance
    mins[i] = minimum(DO_vals_mc)
    if mins[i] < 4.0
        count_fails += 1
    end
end

# original variables
DOr = DOr_orig
CBODw1, NBODw1 = CBODw1_orig, NBODw1_orig
CBOD_w1, NBOD_w1, DO_w1 = CBOD_w1_orig, NBOD_w1_orig, DO_w1_orig

# compute probability of failure and 95% CI
prob_fail = count_fails / N
se = sqrt(prob_fail * (1 - prob_fail) / N)
ci_lo = max(0.0, prob_fail - 1.96 * se)
ci_hi = min(1.0, prob_fail + 1.96 * se)

println("Treated only W1 at ", 100 * p_w1, "% removal")
println("Probability of failing compliance of minimum DO < 4.0 mg/L): ", prob_fail * 100, "%")
println("95% CI: [", ci_lo * 100, "%, ", ci_hi * 100, "%]")
println("For the Monte Carlo sample size of ", N, ", I chose the sample size because 50,000 was a good")
println("balance between speed and accuracy—N=100_000 took much longer to run and a negligible change")
println("while values like 100 and 1_000 were too small and yielded probabilities that fluctuated a lot.")

```

Treated only W1 at 24.81689453125% removal
 Probability of failing compliance of minimum DO < 4.0 mg/L): 2.33%
 95% CI: [2.197770113986285%, 2.462229886013715%]
 For the Monte Carlo sample size of 50000, I chose the sample size because 50,000 was a good
 balance between speed and accuracy—N=100_000 took much longer to run and a negligible change
 while values like 100 and 1_000 were too small and yielded probabilities that fluctuated a lot.

If we treated only W1 at 24.81689453125% removal, the probability of failing compliance of
 minimum DO < 4.0 mg/L is 2.33%, with 95% CI: [2.197770113986285%, 2.462229886013715%].
 For the Monte Carlo sample size of 50000, I chose the sample size because 50,000 was a good
 balance between speed and accuracy—N=100_000 took much longer to run and a negligible
 change in probability, while values like 100 and 1_000 were too small and yielded probabilities
 that fluctuated a lot.

References

List any external references consulted, including classmates.

For problem 1.1 I referenced Prof. Srikrishnan slide deck from the lecture titled “Dissolved Oxygen” to help with the writing of the code and making sure I had the correct model for the Streeter Phelps Equation.

For problem 1.3, I knew I wanted to create a function that could find the compliance threshold number within the array of Waste 1 and Waste 2. I searched it up online if there is a function that can do this and came across the bisection search algorithm from Khan Academy. I used this as my reference and wrote my initial code based on this method. I then consulted with Chat-GPT to make sure I got the formatting right and these are the sources I used for this problem. <https://www.khanacademy.org/computing/computer-science/algorithms/binary-search>

For problem 1.5, I referenced Prof. Srikrishnan’s Monte Carlo lecture slide deck to rehash my understanding of Monte Carlo simulation, and did additional research on Julia documentation online in order to figure out how to correctly calculate CIs.